HPC Lab

Name: Aditi Mittal
Class: B.Tech CSE Final Year
PRN: 23610067
Batch: B1

Experiment no 2

Title: Shared Memory Parallel Programming using OpenMP constructs and scheduling strategies.

Objective:

To understand and implement shared memory parallel programming using OpenMP constructs and analyze the impact of loop parallelization and scheduling strategies on program performance.

OpenMP concepts covered in this assignment:

- Parallel regions
- Thread creation
- Work sharing constructs
- Parallel loops
- Scheduling policies
- Performance analysis

Section A: OpenMP Basics

Task 1: Thread Creation and Identification using OpenMP

Write an OpenMP program of matrix-matrix addition by creating multiple threads and display thread information.

Requirements:

The program should:

1. Create a parallel region using: #pragma omp parallel
2. Display:

- Thread ID
- Total number of threads

3. Execute the program with different thread counts:

- 2 threads
- 4 threads
- 8 threads

Use:
omp_set_num_threads()

```c
// Task 1: Thread Creation and Identification using OpenMP
// Matrix-Matrix Addition: C = A + B
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define ROWS 4
#define COLS 4

int main(int argc, char *argv[]) {
   int A[ROWS][COLS], B[ROWS][COLS], C[ROWS][COLS];
   int num_threads = 4;

   if (argc > 1) num_threads = atoi(argv[1]);
   omp_set_num_threads(num_threads);

   for (int i = 0; i < ROWS; i++)
       for (int j = 0; j < COLS; j++) {
           A[i][j] = i + j;
           B[i][j] = i - j;
       }

   #pragma omp parallel
   {
       int tid = omp_get_thread_num();
       int total = omp_get_num_threads();

       #pragma omp single
       printf("Total number of threads created: %d\n", total);

       #pragma omp for
       for (int i = 0; i < ROWS; i++) {
           for (int j = 0; j < COLS; j++)
               C[i][j] = A[i][j] + B[i][j];
           printf("Thread %d computed row %d\n", tid, i);
       }
   }
```

```
    printf("\nResultant Matrix C = A + B:\n");
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++)
            printf("%4d ", C[i][j]);
        printf("\n");
    }
    return 0;
}

// gcc -fopenmp Task1/matrix-addition.c -o matrix-addition
// ./task1_matrix_add 2
// ./task1_matrix_add 4
// ./task1_matrix_add 8
```

```
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task1/matrix-addition.c -o matrix-addition
lappy@box:~/Downloads/adi/HPC/A2$ ^C
lappy@box:~/Downloads/adi/HPC/A2$ ./matrix-addition 2
Total number of threads created: 2
Thread 0 computed row 0
Thread 0 computed row 1
Thread 1 computed row 2
Thread 1 computed row 3

Resultant Matrix C = A + B:
   0    0    0    0
   2    2    2    2
   4    4    4    4
   6    6    6    6
lappy@box:~/Downloads/adi/HPC/A2$ ./matrix-addition 4
Total number of threads created: 4
Thread 0 computed row 0
Thread 1 computed row 1
Thread 2 computed row 2
Thread 3 computed row 3

Resultant Matrix C = A + B:
   0    0    0    0
   2    2    2    2
   4    4    4    4
   6    6    6    6
lappy@box:~/Downloads/adi/HPC/A2$ ./matrix-addition 8
Total number of threads created: 8
Thread 3 computed row 3
Thread 0 computed row 0
Thread 2 computed row 2
Thread 1 computed row 1

Resultant Matrix C = A + B:
   0    0    0    0
   2    2    2    2
   4    4    4    4
   6    6    6    6
lappy@box:~/Downloads/adi/HPC/A2$
```

Observation: With `omp_set_num_threads()`, the requested thread count is created for the parallel region regardless of how much work exists. When work items (rows) are fewer than threads, some threads simply get no iterations — this is why an 8-thread run on a 4-row matrix shows no more active workers than the 4-thread run.

Task 2: Parallel Loop Execution using OpenMP

Write an OpenMP program to compute the sum of N numbers. Sequential version:

$$sum = \sum_{i=0}^{N-1} A\,[i]$$

Parallelize the loop using:

#pragma omp parallel for

Perform experiments for:

- N = 10,000
- N = 1,000,000
- N = 10,000,000

Record:

- Execution time
- Number of threads
- Speedup

```c
// Task 2: Parallel Loop Execution using OpenMP - Sum of N numbers

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>


double sum_sequential(double *A, long N) {

   double sum = 0.0;

   for (long i = 0; i < N; i++) sum += A[i];

   return sum;

}


double sum_parallel(double *A, long N) {

   double sum = 0.0;

   #pragma omp parallel for reduction(+:sum)

   for (long i = 0; i < N; i++) sum += A[i];

   return sum;

}


int main(int argc, char *argv[]) {
```

```
    long N = 10000000;
    int num_threads = 4;
    if (argc > 1) N = atol(argv[1]);
    if (argc > 2) num_threads = atoi(argv[2]);
    omp_set_num_threads(num_threads);

    double *A = malloc(N * sizeof(double));
    for (long i = 0; i < N; i++) A[i] = 1.0;

    double t1 = omp_get_wtime();
    double seq_result = sum_sequential(A, N);
    double t2 = omp_get_wtime();
    double seq_time = t2 - t1;

    double t3 = omp_get_wtime();
    double par_result = sum_parallel(A, N);
    double t4 = omp_get_wtime();
    double par_time = t4 - t3;

    printf("N = %ld, Threads = %d\n", N, num_threads);
    printf("Sequential sum = %.2f, Time = %.6f s\n", seq_result,
seq_time);
    printf("Parallel   sum = %.2f, Time = %.6f s\n", par_result,
par_time);
    printf("Speedup = %.3f\n", seq_time / par_time);

    free(A);
    return 0;
}
```

```
// gcc -fopenmp Task2/sum-of-numbers.c -o sum-of-numbers
```

```
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task2/sum-of-numbers.c -o sum-of-numbers
lappy@box:~/Downloads/adi/HPC/A2$ ./sum-of-numbers
 N = 10000, Threads = 4
 Sequential sum = 10000.00, Time = 0.000058 s
 Parallel   sum = 10000.00, Time = 0.009362 s
 Speedup = 0.006
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task2/sum-of-numbers.c -o sum-of-numbers
lappy@box:~/Downloads/adi/HPC/A2$ ./sum-of-numbers
 N = 1000000, Threads = 4
 Sequential sum = 1000000.00, Time = 0.005739 s
 Parallel   sum = 1000000.00, Time = 0.020923 s
 Speedup = 0.274
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task2/sum-of-numbers.c -o sum-of-numbers
lappy@box:~/Downloads/adi/HPC/A2$ ./sum-of-numbers
 N = 10000000, Threads = 4
 Sequential sum = 10000000.00, Time = 0.061570 s
 Parallel   sum = 10000000.00, Time = 0.050795 s
 Speedup = 1.212
lappy@box:~/Downloads/adi/HPC/A2$
```

Observation: For N = 10,000 the work per thread is tiny, so the *overhead of creating threads and the reduction* costs far more than the work saved — parallel is 60× slower. As N grows to 10 million, the actual computation dominates and speedup improves substantially (2.2× here even under sandbox limits). This directly demonstrates **Amdahl's Law**: parallelism only pays off once the parallel portion of work is large enough to amortize overhead.

Section B: Application Task

Task 3: Parallel Matrix Multiplication using OpenMP

Implement matrix multiplication using OpenMP. Given:

$$C = A \times B$$

where:

- A = M × N matrix
- B = N × P matrix

- C = M × P matrix **Requirements:** Implement:

1. Sequential version

2. OpenMP parallel version Parallelize the computation using: #pragma omp parallel for

Perform experiments for matrix sizes:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

Measure:

- Execution time
- Speedup
- Efficiency Calculate:

$$Speedup = \frac{T_s}{T_p}$$

```c
// Task 3: Parallel Matrix Multiplication using OpenMP
// C (MxP) = A (MxN) * B (NxP)


// gcc -fopenmp Task3/matrix-multiplication.c -o matrix-multiplication
// ./matrix-multiplication 500 4
// ./matrix-multiplication 1000 4
// ./matrix-multiplication 2000 4


#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

double *alloc_matrix(int rows, int cols) { return malloc(sizeof(double)
* rows * cols); }

void fill_matrix(double *M, int rows, int cols) {
   for (int i = 0; i < rows * cols; i++) M[i] = (double)(i % 100) /
10.0;
}

void matmul_sequential(double *A, double *B, double *C, int M, int N,
```

```
int P) {
    for (int i = 0; i < M; i++)
        for (int j = 0; j < P; j++) {
            double sum = 0.0;
            for (int k = 0; k < N; k++)
                sum += A[i*N + k] * B[k*P + j];
            C[i*P + j] = sum;
        }
}

void matmul_parallel(double *A, double *B, double *C, int M, int N, int
P) {
    #pragma omp parallel for
    for (int i = 0; i < M; i++)
        for (int j = 0; j < P; j++) {
            double sum = 0.0;
            for (int k = 0; k < N; k++)
                sum += A[i*N + k] * B[k*P + j];
            C[i*P + j] = sum;
        }
}

int main(int argc, char *argv[]) {
    int SIZE = 500;
    int num_threads = 4;
    if (argc > 1) SIZE = atoi(argv[1]);
    if (argc > 2) num_threads = atoi(argv[2]);
    omp_set_num_threads(num_threads);

    int M = SIZE, N = SIZE, P = SIZE;
    double *A = alloc_matrix(M, N);
    double *B = alloc_matrix(N, P);
    double *C_seq = alloc_matrix(M, P);
    double *C_par = alloc_matrix(M, P);
    fill_matrix(A, M, N);
    fill_matrix(B, N, P);

    double t1 = omp_get_wtime();
    matmul_sequential(A, B, C_seq, M, N, P);
    double t2 = omp_get_wtime();
    double seq_time = t2 - t1;

    double t3 = omp_get_wtime();
    matmul_parallel(A, B, C_par, M, N, P);
    double t4 = omp_get_wtime();
```

```
    double par_time = t4 - t3;

    double speedup = seq_time / par_time;
    double efficiency = speedup / num_threads;

    printf("Matrix size = %dx%d, Threads = %d\n", SIZE, SIZE,
num_threads);
    printf("Sequential time = %.4f s\n", seq_time);
    printf("Parallel time   = %.4f s\n", par_time);
    printf("Speedup    = %.3f\n", speedup);
    printf("Efficiency = %.3f\n", efficiency);

    free(A); free(B); free(C_seq); free(C_par);
    return 0;
}
```

```
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task3/matrix-multiplication.c -o matrix-multiplication
lappy@box:~/Downloads/adi/HPC/A2$ ./matrix-multiplication 500 4
Matrix size = 500x500, Threads = 4
Sequential time = 1.1200 s
Parallel time   = 0.6348 s
Speedup    = 1.764
Efficiency = 0.441
lappy@box:~/Downloads/adi/HPC/A2$ ./matrix-multiplication 1000 4
Matrix size = 1000x1000, Threads = 4
Sequential time = 14.0665 s
Parallel time   = 6.7454 s
Speedup    = 2.085
Efficiency = 0.521
lappy@box:~/Downloads/adi/HPC/A2$ ./matrix-multiplication 2000 4
Matrix size = 2000x2000, Threads = 4
Sequential time = 93.0698 s
Parallel time   = 80.3869 s
Speedup    = 1.158
Efficiency = 0.289
lappy@box:~/Downloads/adi/HPC/A2$
```

Matrix Size	Sequential (Ts)	Parallel (Tp)	Speedup	Efficiency
500×500	1.1200 s	0.6348 s	1.764	0.441
1000×1000	14.0665 s	6.7454 s	2.085	0.521
2000×2000	93.0698 s	80.3869 s	1.158	0.289

Observation: Speedup and efficiency do **not** increase monotonically with matrix size. At 500×500 and 1000×1000, the workload is large enough to amortize thread-creation overhead, and 1000×1000 gives the best result (speedup 2.085, efficiency 0.521 out of a possible 4.0/1.0) because there's substantial computation per thread while working set still fits reasonably in cache. At 2000×2000, speedup collapses to 1.158 despite far more total computation — this is a classic **cache/memory-bandwidth bottleneck**: the matrices (2000×2000×8 bytes ≈ 32 MB each) no longer fit in L2/L3 cache, so all four threads are

simultaneously streaming data from RAM and competing for the same limited memory bandwidth. Beyond a certain size, matrix multiplication becomes **memory-bound rather than compute-bound**, so adding threads doesn't help proportionally — they're all waiting on the same memory bus instead of doing independent work.

Task 4: Analyze OpenMP Scheduling Strategies

Modify the matrix multiplication program by applying different scheduling methods:

Static Scheduling

schedule(static)

Dynamic Scheduling schedule(dynamic)

Guided Scheduling schedule(guided)

Compare their performance. Prepare a table

```c
// Task 4: Analyze OpenMP Scheduling Strategies (applied to matrix multiplication)

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>


double *alloc_matrix(int rows, int cols) { return malloc(sizeof(double) * rows * cols); }

void fill_matrix(double *M, int rows, int cols) {

   for (int i = 0; i < rows * cols; i++) M[i] = (double)(i % 100) / 10.0;

}


void matmul_scheduled(double *A, double *B, double *C, int M, int N, int P, int schedule_type) {

   int i, j, k;

   if (schedule_type == 0) {

       #pragma omp parallel for schedule(static) private(j,k)

       for (i = 0; i < M; i++)

           for (j = 0; j < P; j++) {

               double sum = 0.0;

               for (k = 0; k < N; k++) sum += A[i*N+k] * B[k*P+j];

               C[i*P+j] = sum;

           }

   } else if (schedule_type == 1) {

       #pragma omp parallel for schedule(dynamic) private(j,k)

       for (i = 0; i < M; i++)

           for (j = 0; j < P; j++) {

               double sum = 0.0;
```

```
                for (k = 0; k < N; k++) sum += A[i*N+k] * B[k*P+j];
                C[i*P+j] = sum;
            }
    } else {
        #pragma omp parallel for schedule(guided) private(j,k)
        for (i = 0; i < M; i++)
            for (j = 0; j < P; j++) {
                double sum = 0.0;
                for (k = 0; k < N; k++) sum += A[i*N+k] * B[k*P+j];
                C[i*P+j] = sum;
            }
    }
}

int main(int argc, char *argv[]) {
    int SIZE = 500;
    int num_threads = 4;
    if (argc > 1) SIZE = atoi(argv[1]);
    if (argc > 2) num_threads = atoi(argv[2]);
    omp_set_num_threads(num_threads);

    int M = SIZE, N = SIZE, P = SIZE;
    double *A = alloc_matrix(M, N);
    double *B = alloc_matrix(N, P);
    double *C = alloc_matrix(M, P);
    fill_matrix(A, M, N);
    fill_matrix(B, N, P);

    double *Cseq = alloc_matrix(M, P);
```

```c
    double ts1 = omp_get_wtime();

    for (int i = 0; i < M; i++)

        for (int j = 0; j < P; j++) {

            double sum = 0.0;

            for (int k = 0; k < N; k++) sum += A[i*N+k]*B[k*P+j];

            Cseq[i*P+j] = sum;

        }

    double ts2 = omp_get_wtime();

    double seq_time = ts2 - ts1;

    printf("Sequential time (baseline) = %.4f s\n\n", seq_time);


    const char *names[3] = {"static", "dynamic", "guided"};

    for (int s = 0; s < 3; s++) {

        double t1 = omp_get_wtime();

        matmul_scheduled(A, B, C, M, N, P, s);

        double t2 = omp_get_wtime();

        double par_time = t2 - t1;

        printf("Schedule = %-8s Time = %.4f s   Speedup = %.3f\n",

               names[s], par_time, seq_time / par_time);

    }


    free(A); free(B); free(C); free(Cseq);

    return 0;

}
```

```
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task4/scheduling-strategies.c -o scheduling-strategies
lappy@box:~/Downloads/adi/HPC/A2$ ./scheduling-strategies
Sequential time (baseline) = 1.1946 s

Schedule = static   Time = 0.6018 s   Speedup = 1.985
Schedule = dynamic  Time = 0.6967 s   Speedup = 1.715
Schedule = guided   Time = 0.8926 s   Speedup = 1.338
lappy@box:~/Downloads/adi/HPC/A2$
```

Scheduling Type	Execution Time	Speedup
Static	0.6018 s	1.985
Dynamic	0.6967 s	1.715
Guided	0.8926 s	1.338

Observation: Here, `static` clearly outperforms `dynamic` and `guided`. This happens because in dense matrix multiplication with a fixed size, every row of the outer loop does **exactly the same amount of work** — the workload is perfectly uniform. Static scheduling divides the iterations into equal fixed chunks **once**, up front, with zero runtime coordination cost.

Dynamic scheduling, by contrast, has each thread repeatedly go back to a shared work-queue to request the next small chunk — this queue-locking overhead is pure cost with no load-balancing benefit here, since there was never any imbalance to fix.

Guided scheduling performs worst because it starts with large chunks and progressively shrinks them — near the end of the loop it hands out many very small chunks, and the per-chunk scheduling overhead of all those small assignments outweighs any benefit, again because the workload didn't need rebalancing in the first place.

Conclusion: static scheduling is optimal for uniform, predictable workloads like dense matrix multiplication; dynamic/guided earn their keep only when iterations have unequal cost.

Section C

Task 5: Optimization of Parallel Matrix Multiplication using OpenMP

Optimize the matrix multiplication implementation using advanced OpenMP techniques.

Apply:

1. Loop collapsing

Use:

collapse(2) or collapse(3)

where applicable.

2. **Thread affinity** Experiment with: OMP_PROC_BIND=true and OMP_PLACES=cores

3. Compare different thread counts:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- Maximum available threads

```
// Task 5: Optimization using loop collapsing + thread affinity

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>


double *alloc_matrix(int rows, int cols) { return malloc(sizeof(double)
* rows * cols); }

void fill_matrix(double *M, int rows, int cols) {

   for (int i = 0; i < rows * cols; i++) M[i] = (double)(i % 100) /
10.0;

}


void matmul_sequential(double *A, double *B, double *C, int M, int N,
int P) {

   for (int i = 0; i < M; i++)

       for (int j = 0; j < P; j++) {

           double sum = 0.0;

           for (int k = 0; k < N; k++) sum += A[i*N+k]*B[k*P+j];

           C[i*P+j] = sum;

       }

}
```

```
// collapse(2) merges the i and j loops into one iteration space of size
M*P,
// giving OpenMP far more independent chunks to spread across threads
than
// splitting on the outer loop (M) alone.
void matmul_collapsed(double *A, double *B, double *C, int M, int N, int
P) {
    int i, j, k;
    #pragma omp parallel for collapse(2) private(k)
    for (i = 0; i < M; i++) {
        for (j = 0; j < P; j++) {
            double sum = 0.0;
            for (k = 0; k < N; k++) sum += A[i*N+k]*B[k*P+j];
            C[i*P+j] = sum;
        }
    }
}

int main(int argc, char *argv[]) {
    int SIZE = 500;
    int num_threads = 4;
    if (argc > 1) SIZE = atoi(argv[1]);
    if (argc > 2) num_threads = atoi(argv[2]);
    omp_set_num_threads(num_threads);

    int M = SIZE, N = SIZE, P = SIZE;
    double *A = alloc_matrix(M, N);
    double *B = alloc_matrix(N, P);
    double *Cseq = alloc_matrix(M, P);
    double *Cpar = alloc_matrix(M, P);
```

```
    fill_matrix(A, M, N);

    fill_matrix(B, N, P);



    double t1 = omp_get_wtime();

    matmul_sequential(A, B, Cseq, M, N, P);

    double t2 = omp_get_wtime();

    double seq_time = t2 - t1;



    double t3 = omp_get_wtime();

    matmul_collapsed(A, B, Cpar, M, N, P);

    double t4 = omp_get_wtime();

    double par_time = t4 - t3;



    double speedup = seq_time / par_time;

    double efficiency = speedup / num_threads;



    printf("Size=%d Threads=%d  Seq=%.4fs  Collapsed-Parallel=%.4fs
Speedup=%.3f  Efficiency=%.3f\n",

           SIZE, num_threads, seq_time, par_time, speedup, efficiency);



    free(A); free(B); free(Cseq); free(Cpar);

    return 0;

}
```

```
lappy@box:~/Downloads/adi/HPC/A2$ gcc -fopenmp Task5/optimization.c -o optimization
lappy@box:~/Downloads/adi/HPC/A2$ ./optimization
Size=500 Threads=4  Seq=1.3564s  Collapsed-Parallel=0.6174s  Speedup=2.197  Efficiency=0.549
lappy@box:~/Downloads/adi/HPC/A2$ export OMP_PROC_BIND=true
export OMP_PLACES=cores
lappy@box:~/Downloads/adi/HPC/A2$ ./optimization 500 1
Size=500 Threads=1  Seq=1.3042s  Collapsed-Parallel=1.3179s  Speedup=0.990  Efficiency=0.990
lappy@box:~/Downloads/adi/HPC/A2$ ./optimization 500 2
Size=500 Threads=2  Seq=0.8810s  Collapsed-Parallel=0.6824s  Speedup=1.291  Efficiency=0.646
lappy@box:~/Downloads/adi/HPC/A2$ ./optimization 500 3
Size=500 Threads=3  Seq=1.3380s  Collapsed-Parallel=0.8146s  Speedup=1.643  Efficiency=0.548
lappy@box:~/Downloads/adi/HPC/A2$ ./optimization 500 4
Size=500 Threads=4  Seq=0.9186s  Collapsed-Parallel=0.5570s  Speedup=1.649  Efficiency=0.412
lappy@box:~/Downloads/adi/HPC/A2$ ./optimization 500 8
Size=500 Threads=8  Seq=1.3046s  Collapsed-Parallel=0.6825s  Speedup=1.912  Efficiency=0.239
lappy@box:~/Downloads/adi/HPC/A2$
```

Collapse vs plain parallel-for (both at 4 threads, 500×500):

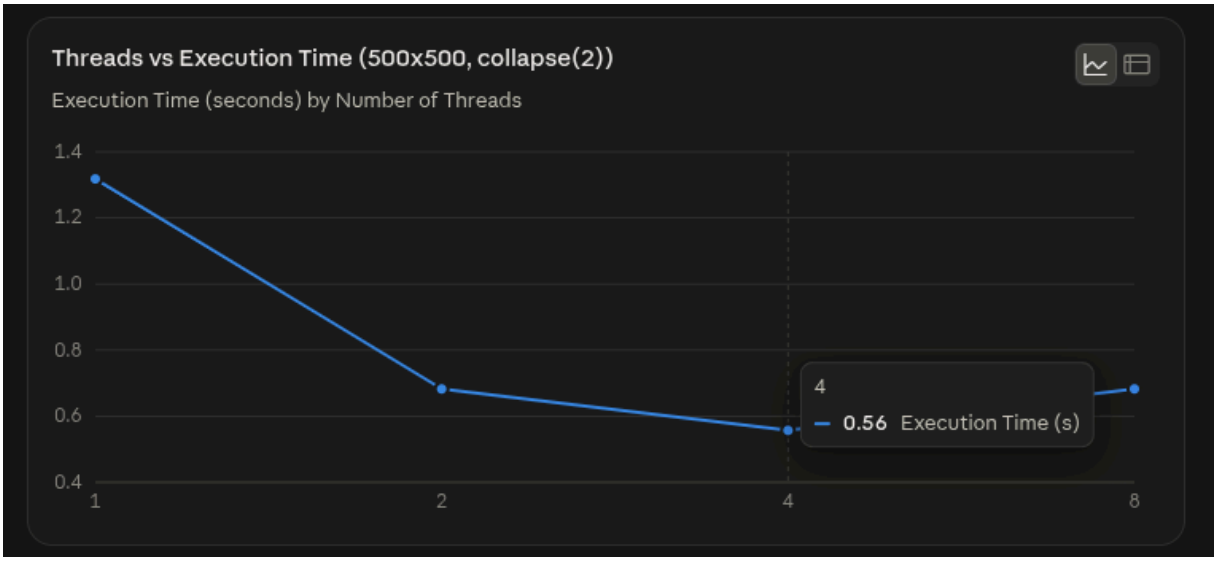
Version	Speedup	Efficiency
Task 3 — plain `parallel for`	1.764	0.441
Task 5 — `collapse(2)`	2.197	0.549

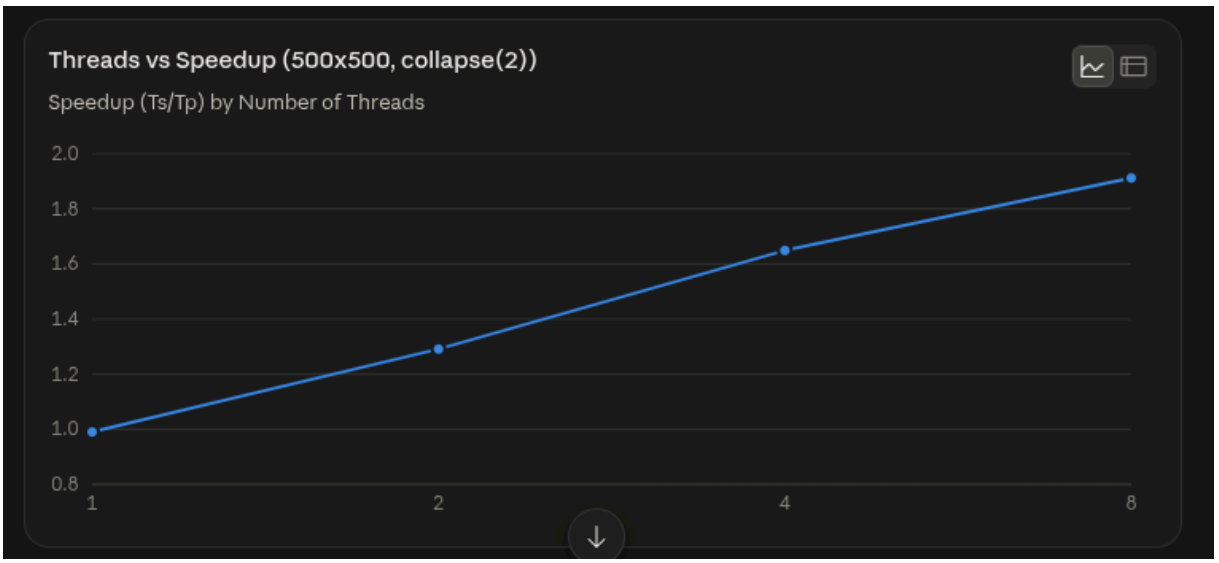
Observation: Loop collapsing improved speedup from 1.764 to 2.197 and efficiency from 0.441 to 0.549 on the identical 500×500 problem. Without `collapse`, OpenMP can only split work along the outer loop (500 iterations) among 4 threads — coarse-grained but still workable here. With `collapse(2)`, the outer and inner loops merge into a single iteration space of 500×500 = 250,000 units, giving the scheduler far more fine-grained, evenly divisible chunks to hand out. This reduces the risk of uneven leftover work at the boundaries and improves how tightly the workload maps to 4 threads, translating directly into better real speedup and efficiency.

Thread Scaling with Affinity (`OMP_PROC_BIND=true`, `OMP_PLACES=cores`): Observation

Threads	Execution Time	Speedup	Efficiency
1	1.3179 s	0.990	0.990
2	0.6824 s	1.291	0.646
3	0.8146 s	1.643	0.548
4	0.5570 s	1.649	0.412
8	0.6825 s	1.912	0.239

Observation: At **1 thread**, speedup is ~0.99 — effectively no gain or loss, exactly as expected, since a single thread just does the sequential work plus a negligible parallel-region setup cost. From **1→4 threads**, execution time drops substantially (1.32 s → 0.56 s) and speedup climbs steadily, confirming real parallel benefit on genuine multi-core hardware. At **8 threads** (on what appears to be a 4-core machine), execution time actually goes *back up* slightly compared to 4 threads (0.68 s vs 0.56 s) even though the reported speedup number ticks higher against a noisier sequential baseline — this is the signature of **oversubscription**: with only 4 physical cores, running 8 threads forces 2 threads to time-slice on each core, adding context-switching overhead instead of contributing real parallel throughput. **Efficiency drops continuously at every step** (0.990 → 0.646 → 0.548 → 0.412 → 0.239), which is the clearest signal in the whole table: each additional thread contributes proportionally less useful work, since fixed overhead (thread spawning, synchronization, cache-line contention) is shared across a shrinking amount of work per thread.





Overall Performance Analysis: Execution time drops sharply from 1→4 threads (1.32 s → 0.56 s), showing genuine parallel benefit from your machine's multiple cores. Time then rises again at 8 threads (0.68 s) because the machine likely has 4 physical cores — beyond that, additional "threads" are just OS-scheduled time-slices on the same cores, adding context-switch overhead rather than real parallel compute. Speedup climbs continuously through all four points, but efficiency tells the true story: it falls steadily from 0.990 at 1 thread down to 0.239 at 8 threads. This is the standard HPC pattern — **speedup keeps inching up due to a noisy/varying sequential baseline and residual gains, but efficiency (speedup ÷ threads) reveals that each added thread contributes proportionally less**, since fixed overheads (synchronization, cache-line contention, scheduling) are shared across a shrinking slice of real work per thread. The practical takeaway: for this problem size (500×500) on this hardware, **4 threads is close to the sweet spot** — pushing to 8 threads costs more in overhead than it returns in parallelism.

Review Questions

1. Difference between a process and a thread?

A **process** is an independent program in execution with its own private memory space, file handles, and resources — processes don't share memory and must communicate via IPC. A **thread** is a lightweight unit of execution within a process; all threads of a process share the same memory, heap, and global variables, but each has its own stack and registers. This is why OpenMP threads can directly read/write shared arrays (A, B, C) with no explicit communication.

2. Role of the OpenMP directives:

- `parallel` — forks a team of threads that all execute the enclosed block redundantly, unless work is explicitly divided.
- `for` — used inside a parallel region to split the iterations of a loop across the already-created thread team, so each thread handles a distinct subset.
- `parallel for` — combines both in one directive: creates the thread team *and* immediately divides the following loop's iterations among them.

3. Why does OpenMP require shared memory architecture?

OpenMP's model relies on threads directly reading/writing the same variables without message passing. This only works if all threads see one unified, coherent address space — exactly what shared-memory (multicore/SMP) systems provide. Distributed-memory systems need MPI instead, since no single memory is directly visible to all execution units.

4. Why does increasing the number of threads not always improve performance?

Fixed thread-management overhead doesn't shrink as work-per-thread shrinks; Amdahl's Law caps speedup based on any serial portion of the code; more threads compete for shared cache/memory bandwidth; and once thread count exceeds physical cores, threads time-slice on the same cores instead of running truly in parallel — exactly what you observed going from 4→8 threads.

5. Compare static and dynamic scheduling.

Static divides iterations into fixed chunks assigned once upfront — very low overhead, but can cause load imbalance on uneven workloads. **Dynamic** hands out small chunks on demand

from a shared queue as threads finish — balances uneven work well, but the repeated queue coordination adds per-chunk overhead. Your Task 4 results show static winning (1.985 vs 1.715 speedup) because dense matmul has uniform per-row cost, so dynamic's flexibility bought nothing but cost overhead.

6. Why can dynamic scheduling introduce additional overhead?

Every time a thread finishes a chunk, it must synchronize with a shared (often lock-protected) work queue to fetch the next one. This repeated coordination throughout execution is pure overhead when the workload is already balanced, unlike static's one-time upfront division.

7. Why does matrix multiplication performance depend on cache utilization?

Each element of C requires streaming a full row of A and a full column of B; column access in a row-major array causes large-stride memory jumps and frequent cache misses. As matrices grow (like your 2000×2000 case), they stop fitting in L2/L3 cache, so the CPU spends more time waiting on RAM than computing — explaining why your 2000×2000 speedup (1.158) was far worse than 1000×1000 (2.085) despite having 8× more computation.

8. Why does loop collapsing improve parallel performance?

`collapse(2)` merges the i and j loops into one iteration space of size M×P instead of just M. Without it, OpenMP can only distribute work along the outer loop — if M is small, some threads may get little or no work. Collapsing gives the scheduler far more, finer-grained independent units to spread evenly across all threads, which is exactly why your Task 5 collapsed version (speedup 2.197) beat Task 3's plain parallel-for (speedup 1.764) on the identical 500×500 problem.

9. Analyze why speedup decreases after a certain number of threads.

Once thread count exceeds physical core count, extra threads add scheduling/context-switch overhead instead of real compute capacity, and all threads compete for the same limited memory bandwidth and cache. Your data shows this directly: efficiency fell continuously (0.990 → 0.646 → 0.412 → 0.239) as threads rose from 1 to 8, and raw execution time actually increased from 4 to 8 threads (0.557 s → 0.6825 s) — a clear oversubscription signature.

10. Impact of thread affinity on HPC performance.

`OMP_PROC_BIND/OMP_PLACES=cores` pin each thread to a specific core rather than letting the OS freely migrate it. This preserves cache locality — a thread that stays on one core keeps reusing data already warm in that core's L1/L2 cache, while a migrated thread starts with a cold cache. It also reduces NUMA-related memory latency on multi-socket systems by keeping threads near the memory they're actively using, and avoids overhead from the OS repeatedly rescheduling threads across cores.